

# DOCUMENTAÇÃO E EXPLICAÇÃO: PROJETO DE MONITORAMENTO DE NÍVEIS DE ÁGUA

## OBJETIVO DO PROJETO

Desenvolver um sistema simples em Python para monitoramento de reservatórios de água, com exibição de mensagens de alerta coloridas no terminal, utilizando a biblioteca 'colorama'.

## ESTRUTURA DO CÓDIGO E EXPLICAÇÃO PASSO A PASSO

### 1. IMPORTAÇÃO E INICIALIZAÇÃO

```
from colorama import Fore, Style, init
init(autoreset=True)
```

- Explicação: Importamos as classes 'Fore' (cores do texto), 'Style' (estilos como brilho) e 'init'.
- O comando 'init(autoreset=True)' é crucial. Ele configura o terminal para que, após cada comando 'print' com cor, o estilo seja restaurado automaticamente. Isso evita que o restante do terminal fique colorido após a execução.

### 2. ESTRUTURA DE DADOS (LISTA DE DICIONÁRIOS)

```
reservatorio_niveis = [
    {"nivel": 1, "situacao": "Muito baixo (crítico)", "cor": Fore.RED},
    ...
]
```

- Explicação: Utilizamos uma Lista que contém Dicionários. Cada dicionário representa um nível do reservatório.
- Vantagem: Centralizamos os dados. Se precisarmos adicionar um "Nível 6" ou mudar a cor de um alerta, alteramos apenas nesta estrutura, sem precisar mexer na lógica do programa.

### 3. FUNÇÃO DE EXIBIÇÃO (LÓGICA)

```
def exibir_mensagem(nivel_informado):
    for item in reservatorio_niveis:
        if item["nivel"] == nivel_informado:
            print(f"{item['cor']}[Nível {item['nivel']}] - Situação: {item['situacao']}")
            return
```

- Explicação:
  - A função recebe um número (nivel\_informado).
  - O loop 'for' percorre nossa lista.
  - O 'if' compara se o nível atual da lista é o que buscamos.
  - A 'f-string' (o texto com o 'f' antes das aspas) permite inserir variáveis diretamente nas chaves '{}'. O código 'item['cor']' aplica a cor no texto dinamicamente.
  - O 'return' encerra a função ao encontrar o item, economizando processamento.

### 4. MOTOR DA SIMULAÇÃO

```
def simular_sistema():
    for i in range(1, 6):
        exibir_mensagem(i)
```

- Explicação: O 'range(1, 6)' gera os números 1, 2, 3, 4 e 5. O loop chama nossa função de exibição para cada um desses números, automatizando o teste do sistema.

### 5. EXECUÇÃO DO PROGRAMA

```
if __name__ == "__main__":
    simular_sistema()
```

- Explicação: Esta é uma prática recomendada (Best Practice). Garante que a função 'simular\_sistema' só seja executada se este arquivo for rodado diretamente, e não se ele for importado como um módulo em outro programa.

#### POR QUE ESTE CÓDIGO ATENDE AO ENUNCIADO DA ETEC?

- Utilização de listas: Sim, a lista de dicionários armazena todos os estados.
- Biblioteca colorama: Sim, foi integrada e configurada.
- Função para definir cor/mensagem: Sim, a função 'exibir\_mensagem' gerencia a lógica de exibição.
- Exibição no terminal: O código utiliza 'print' formatado para o console.
- Restauração do estilo: O 'autoreset=True' atende a este requisito de forma eficiente.
- Sem entrada de dados do usuário: O 'range' simula o monitoramento de forma contínua e automatizada.

#### DICAS PARA O ESTUDO

- Se perguntarem sobre o 'autoreset=True', explique que ele garante a integridade visual do terminal, evitando "vazamento" de cores para outros programas ou comandos subsequentes.
- Se perguntarem sobre a escolha da lista de dicionários, destaque a "manutenibilidade" do código (facilidade de manutenção).

c